

LibTorch Day 10

Understanding LibTorch to build a neural network playground!

Here's architectural breakdown of `torch::nn`

1. The Core Processors (Feature Extractors) `torch::nn::Linear (in_features, out_features)`

- What is it : A brute-force connector. It connects every single input point to every single output point.
- When to use it : When your data has no spatial or sequential relationship. It is used at the very end of a network to map abstract features to final decisions.
- Why : It calculates the weighted sum of all inputs.
- The Shape Change : If you pass a batch of 32 flat arrays containing 512 numbers `[32, 512]` into Linear layer set to output 10 numbers. The output shape becomes `[32, 10]`.

`torch::nn::Conv2d (in_channels, out_channels, kernel_size)`

- What is it : The spatial scanner. Instead of looking at everything at once, it slides a small window across an image to find patterns like images, edges, corners, or specific shapes.

- When to use it : For any 2D spatial data like camera frames, LiDAR bird's-eye view costmaps, or spectrograms.
- Why : It preserves spatial relationships. A wheel is a circle regardless of whether it is the top, left or bottom right of the image.
- The Shape Change : Passing a batch of 8 RGB images $[8, 3, 224, 224]$ through a Conv2d layer with 16 output channels transforms the shape to $[8, 16, 224, 224]$ (assuming padding is used). Deepens the data to extract more features.

2. The Sequential Memory (Time-Series)

`torch.nn.LSTM` (Long Short-Term Memory)

&

`torch.nn.GRU` (Gated Recurrent Unit)

- What they are : Memory modules. Unlike Linear or Conv2d layers which forget everything after one frame, these layers maintain an internal "hidden state" that remembers previous inputs.
- When to use them : When processing sequential data over time. In robotics, this is trajectory prediction, sensor fusion over time or speech recognition.

- Why: If a ball rolls behind a box, a normal CNN thinks the ball disappeared. An LSTM remembers the ball's trajectory and knows it is still there. GRU is simply a faster, more lightweight version of LSTM that uses less memory.
- The Shape Change: You pass in a batch of 10 sequences, where each sequence is 50 timesteps long, with 4 sensor readings per step $[10, 50, 4]$. The output will be the processed features for each time step $[10, 50, \text{hidden-size}]$.

3. The Stabilizers and Regularizers

`torch.nn.BatchNorm2d(num-features)`

- What it is: The mathematical stabilizer. It forces the output of the previous layer to have a mean of 0 and a standard deviation of 1.
- When to use it: Immediately after a Conv2d layer.
- Why: Without it, the math can easily explode into massive numbers or shrink to zero as data passes through dozens of layers.

It keeps the data distribution perfectly centered, allowing the network to train much faster.

- Shape change: None. Input $[8, 16, 224, 224]$
Outputs $[8, 16, 224, 224]$.

torch.nn.Dropout(p)

- What is it : The anti-memorization tool. During training, it randomly turns off a percentage p of the neurons.
- When to use it : Usually placed between Linear layers.
- Why : If you do not use Dropout, the network might just memorize the training dataset instead of actually learning the underlying patterns.
- Shape change : None